JSXGraph

JSXGraph is a JavaScript library for interactive geometry, function plotting, charting, and data visualization in a web browser. It is widely used in educational contexts to create dynamic mathematical visualizations.

For more information, visit the JSXGraph website.

- **Try it on LiaScript:**

  https://liascript.github.io/course/?https://raw.githubusercontent.com/liaTemplates/JSXGraph/main/README.md

- **See the project on Github:**

  https://github.com/liaTemplates/JSXGraph

- **Experiment in the LiveEditor:**

  https://liascript.github.io/LiveEditor/?/show/file/https://raw.githubusercontent.com/liaTemplates/JSXGraph/main/README.md

Like with other LiaScript templates, there are three ways to integrate JSXGraph, but the easiest way is to copy the import statement into your project. For more information, see the Sec. Implementation.

1. Load the latest macros via (this might cause breaking changes)

   `import: https://raw.githubusercontent.com/liaTemplates/JSXGraph/main/README.md`

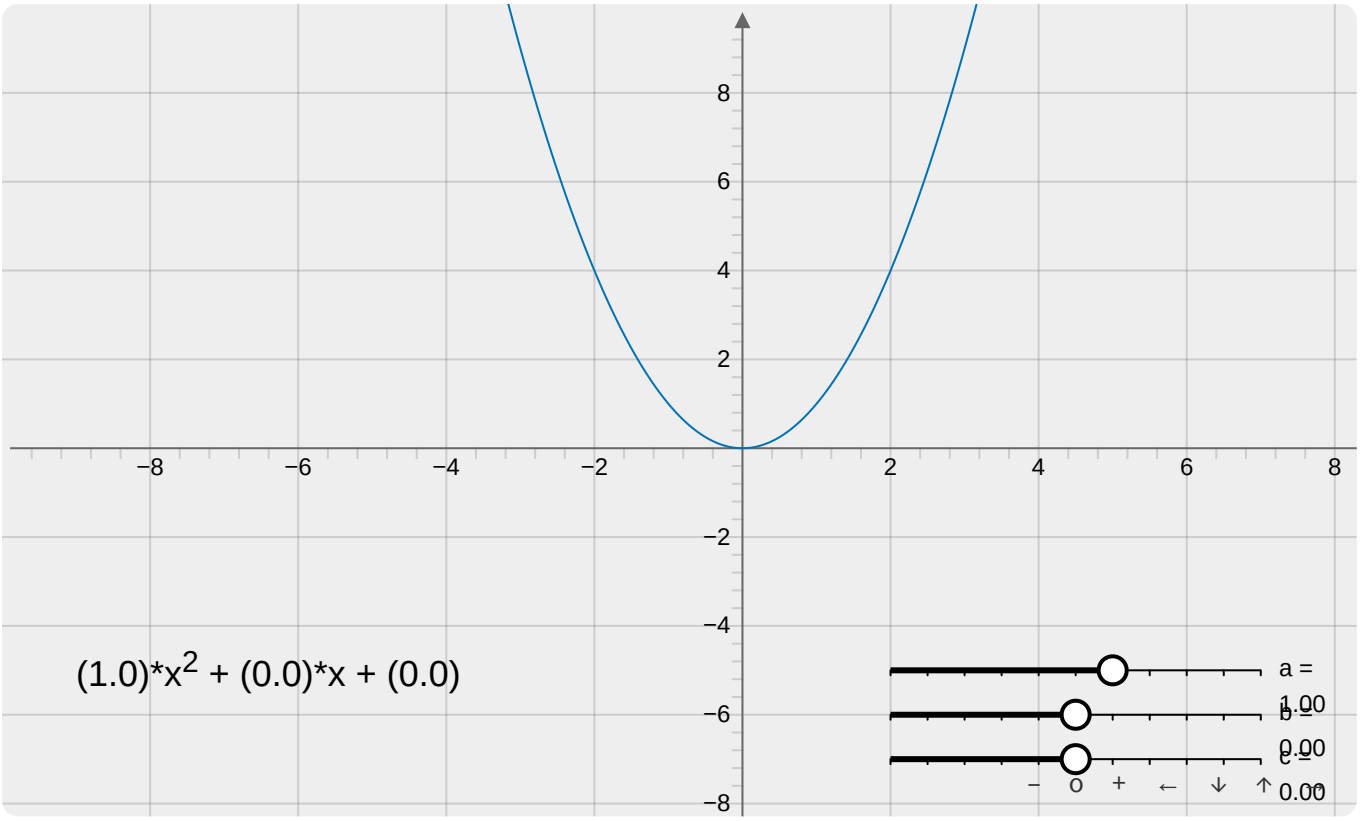
   or the current version 0.0.3 via:

   `import: https://raw.githubusercontent.com/LiaTemplates/JSXGraph/0.0.3/README.md`

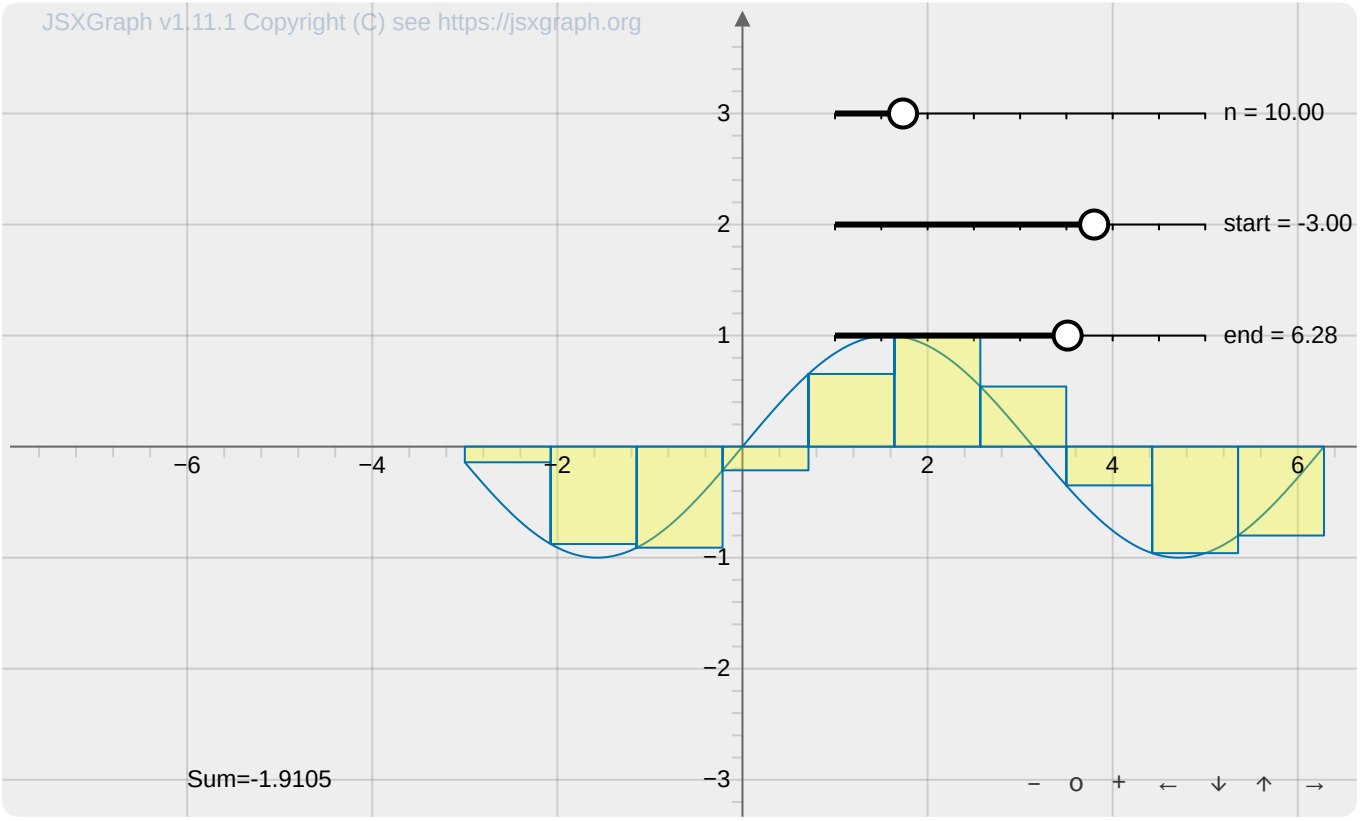
2. **Copy the definitions into your Project**
3. Clone this repository on GitHub

`@JSX.Graph`

This is the most common way to create JSXGraph graphs in LiaScript. It allows you to create a JSXGraph board with the default parameters. The board is created with the default bounding box and axis settings. You can use the `@JSX.Graph.withParams` macro to customize the board parameters.

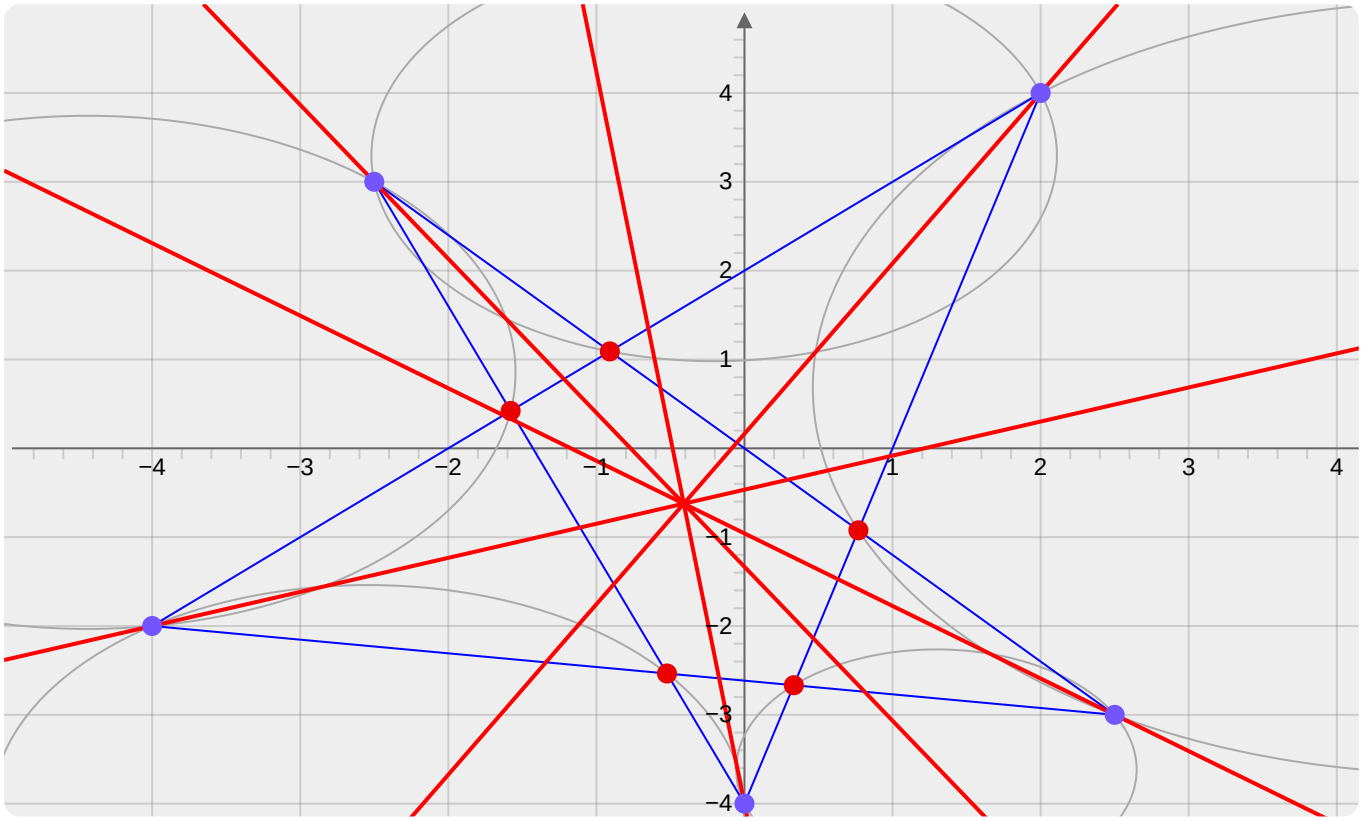


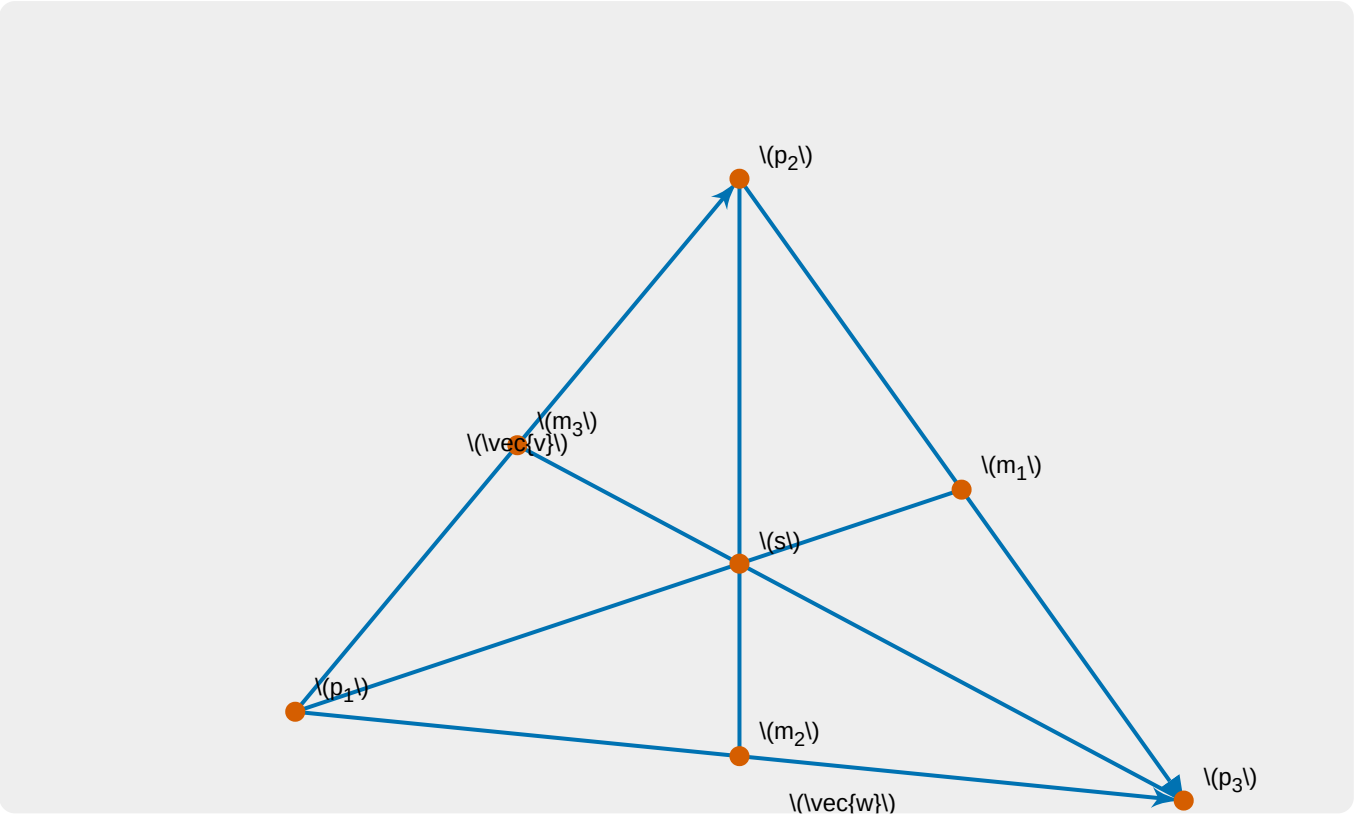
Optionally, you can also manually change the parameters of the board, therefor the board has to be named `board` and is not newly created, but overwritten. And the element it is placed, is called `jxgbox` or use `BOARDID`:



`@JSX.Graph.withParams`

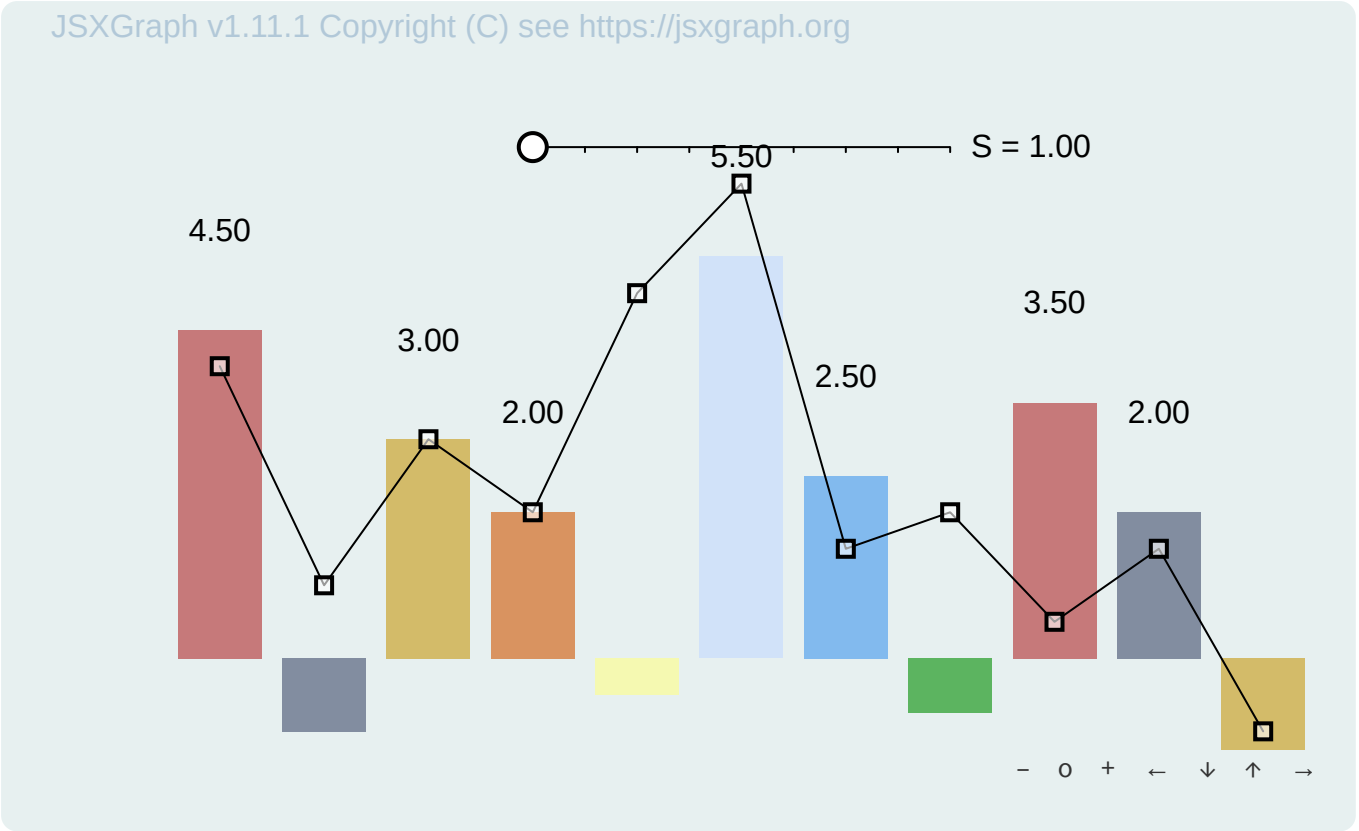
This macro allows you to create a JSXGraph board with custom parameters. You can specify the bounding box, axis visibility, and other parameters directly in the macro.





@JSX.Script

This macro allows your users to read and modify the code that is used to create the JSXGraph board. It is useful for educational purposes, where you want to show the underlying code and allow users to experiment with it. The difference is indicated by the different background color of the code block. Simply double-click onto the board, to reveal the code and modify it.



@JSX.Eval

This macro turns a code block into an executable JSXGraph code block. It is useful for quickly testing and experimenting with JSXGraph code snippets. Simply write your JSXGraph code in a code block and prefix and append `@JSX.Eval`.

```
board = JXG.JSXGraph.initBoard(jxgbox, {
  boundingbox: [-5, 5, 5, -5],
  axis: false,
  showNavigation: false,
  zoom: {
    enabled: false
  },
  pan: {
    enabled: false
  }
});

var view = board.create(
  'view3d',
  [[-2.5, -4], [5, 8], [[-2, 2], [-2, 2], [-2, 2]]],
  {
    projection: 'central',
    trackball: {enabled: true},
    depthOrder: {enabled: true},
    xAxis: {visible: false},
```

```
    yAxis: {visible: false},
    zAxis: {visible: false},
    xPlaneRear: {visible: false},
    yPlaneRear: {visible: false},
    zPlaneRear: {fillOpacity: 0.2, visible: false}
});

let rho = 1.6180339887;
let vertexList = [
  [0, -1, -rho], [0, +1, -rho], [0, -1, rho], [0, +1, rho],
  [1, rho, 0], [-1, rho, 0], [1, -rho, 0], [-1, -rho, 0],
  [-rho, 0, 1], [-rho, 0, -1], [rho, 0, 1], [rho, 0, -1]
];
let faceArray = [
  [4, 1, 11],
  [11, 1, 0],
  [6, 11, 0],
  [0, 1, 9],
  [11, 10, 4],
  [9, 1, 5],
  [8, 9, 5],
  [5, 3, 8],
  [6, 10, 11],
  [2, 3, 10],
  [2, 10, 6],
  [8, 3, 2],
  [3, 4, 10],
  [7, 8, 2],
  [9, 8, 7],
  [0, 9, 7],
  [4, 3, 5],
  [5, 1, 4],
  [0, 7, 6],
  [7, 2, 6]
];

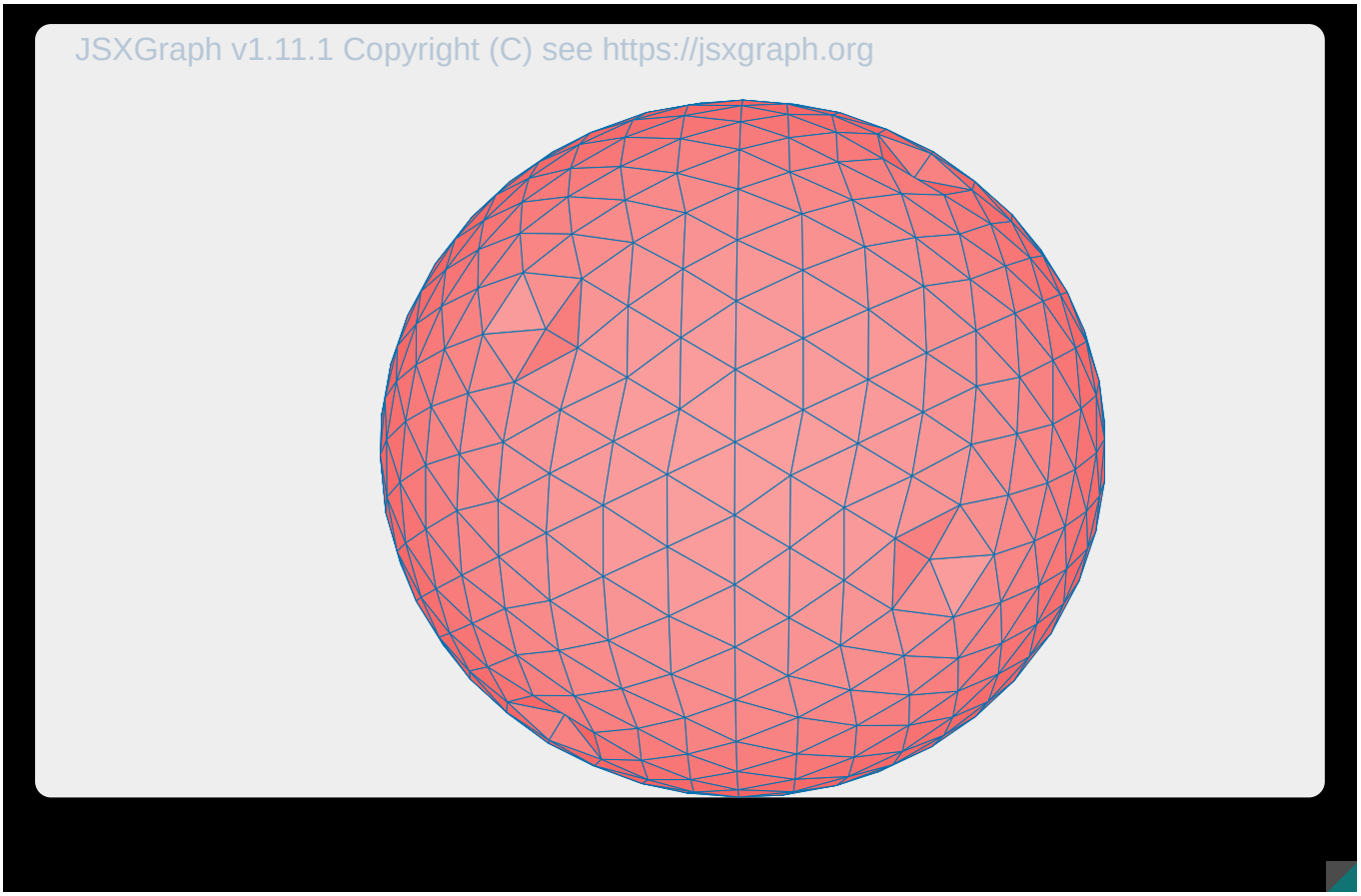
let iterations = 3;    // 0 is a 20 sided icosahedron,  1 is 80 sided
  320 sided, 3 is 1280 sided, etc

/* midpoint between two vertices */
let midPoint = (p1, p2) => [(p2[0] + p1[0]) / 2, (p2[1] + p1[1]) / 2
  (p2[2] + p1[2]) / 2];

/* calculate the unit vector */
let v3Unit = (v) => {
  let length = Math.sqrt(Math.pow(v[0], 2) + Math.pow(v[1], 2) + Mat
    (v[2], 2));
  return [v[0] / length, v[1] / length, v[2] / length];
};

let newFaceArray = [];
```

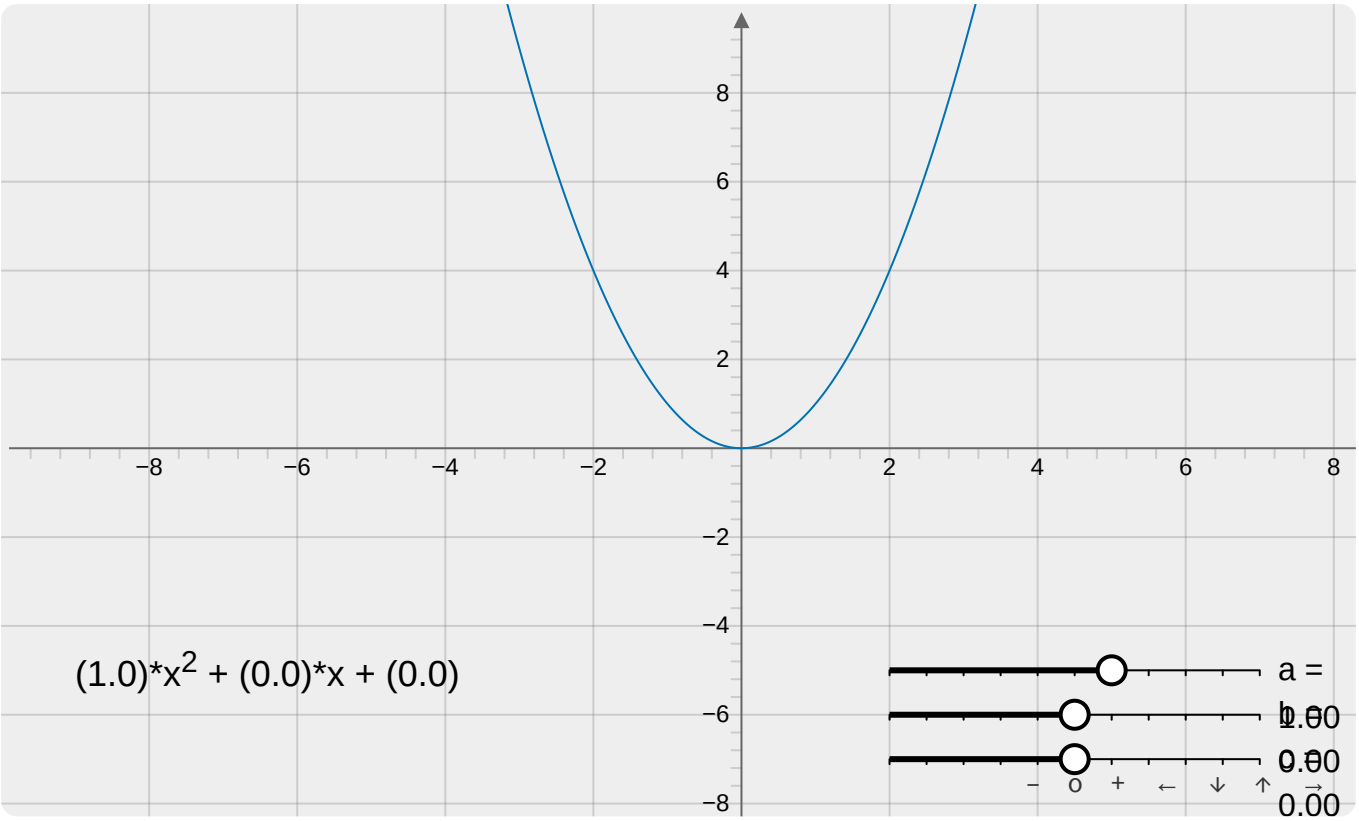
```
69
70  for (let j = 0; j < iterations; j++) {
71    newFaceArray = [];
72    for (let i = 0; i < faceArray.length; i++) {
73      let f = faceArray[i];
74
75      // add three new points at the midpoint of each vertex
76      let m0 = midPoint(vertexList[f[1]], vertexList[f[2]]);
77      let m1 = midPoint(vertexList[f[0]], vertexList[f[2]]);
78      let m2 = midPoint(vertexList[f[0]], vertexList[f[1]]);
79
80      let radius = 2;  // our icos ran from +1 to -1
81
82      let p0 = vertexList.push(v3Unit(m0).map((n) => n * radius)) - 1;
          //new vertex point on surface of sphere through centroid and o
83      let p1 = vertexList.push(v3Unit(m1).map((n) => n * radius)) - 1;
84      let p2 = vertexList.push(v3Unit(m2).map((n) => n * radius)) - 1;
85
86      // now four faces - the three corner-to-midpoints and then all t
          midpoints
87      newFaceArray.push([f[0], p2, p1]);
88      newFaceArray.push([f[1], p0, p2]);
89      newFaceArray.push([f[2], p1, p0]);
90      newFaceArray.push([p0, p1, p2]);
91    }
92    faceArray = newFaceArray;   // in case we go around again
93  }
94
i 95  var ico = view.create('polyhedron3d', [vertexList, faceArray], {
96    fillColorArray: [],
97    fillOpacity: 1,
98    strokeWidth: 0.1,
99    layer: 12,
100   shader: {
101     enabled: true,
102     type: 'angle',
103     hue: 0,
104     saturation: 90,
105     minlightness: 60,
106     maxLightness: 80
107   }
108 });
```



@JSX.Load

This macro allows you to load JSXGraph code from an external file. It is useful for organizing your code and reusing it across multiple projects. Simply provide the URL of the file containing the JSXGraph code.

```
@[JSX.Load](test.js)
```



Implementation

The LiaScript implementation of JSXGraph is based on a custom webcomponent `<jsx-graph>`. See the implementation in src/index.ts for more details.

```
script:  dist/index.js

@JSX.Graph
<jsx-graph>
@0
</jsx-graph>
@end

@JSX.Graph.withParams
<jsx-graph @0>
@1
</jsx-graph>
@end

@JSX.Script
<script run-once modify="//jsx\n" style="display: block">
const code = `//jsx
@0//jsx
`;

"HTML: <jsx-graph background='false'>" + code + "</jsx-graph>";
</script>
```

```
</script>
@end

@JSX.Eval: <script>console.html(`<jsx-graph>@input</jsx-graph>`); ""</scrip

@JSX.Load: <script style="display: block; width: 100%" modify="false" run-o
  >fetch("@0").then(r => r.text()).then(t => send.lia(`HTML: <jsx-graph>${t
  -graph>`)); "LIA: wait"</script>
```